OCR H446 / PAPER 2 / UNIT 2.3

Algorithms

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

01 Design an algorithm from a brief	8
02 Choose an algorithm for the situation	12
03 Analyse time and space costs	16
04 Recognise constant, log and linear	20
05 Compare polynomial and exponential	24
06 Trace a stack: LIFO	28
07 Trace a queue: FIFO	32
08 Follow a linked list	36
09 Search a binary search tree	40
10 Depth-first post-order traversal	44
11 Breadth-first traversal of a tree	48
12 Linear search	52
13 Binary search	56
14 Bubble sort	60
15 Insertion sort	64
16 Merge sort	68
17 Quick sort	72

OCR H446 / PAPER 2 / UNIT 2.3

Algorithms

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

18 Dijkstra: shortest known distances	76
19 A*: use a lower-bound heuristic	80
20 Synoptic algorithm decisions	84
Write searches in OCR-style pseudocode	89
Implement stack, queue and linked-list operations	93
Write post-order DFS and queue-based BFS	97
Read and write bubble and insertion sort	101
Trace recursive merge and quick sort	105
Work a Dijkstra and A* route trace	109

2.3 / MASTER CONTENTS

Choose a learning cycle

Each topic has one concept page, one worked model, exam-style practice and a separate answer page.

2.3.1(a)	Design an algorithm from a brief	7-10
2.3.1(a,b)	Choose an algorithm for the situation	11-14
2.3.1(b,c)	Analyse time and space costs	15-18
2.3.1(c,d)	Recognise constant, log and linear	19-22
2.3.1(c,d)	Compare polynomial and exponential	23-26
2.3.1(e)	Trace a stack: LIFO	27-30
2.3.1(e)	Trace a queue: FIFO	31-34
2.3.1(e)	Follow a linked list	35-38
2.3.1(e)	Search a binary search tree	39-42
2.3.1(e)	Depth-first post-order traversal	43-46

NAVIGATION

2.3 / MASTER CONTENTS

Choose a learning cycle

Each topic has one concept page, one worked model, exam-style practice and a separate answer page.

2.3.1(e)	**Breadth-first traversal of a tree**	47-50
2.3.1(f)	**Linear search**	51-54
2.3.1(f)	**Binary search**	55-58
2.3.1(f)	**Bubble sort**	59-62
2.3.1(f)	**Insertion sort**	63-66
2.3.1(f)	**Merge sort**	67-70
2.3.1(f)	**Quick sort**	71-74
2.3.1(f)	**Dijkstra: shortest known distances**	75-78
2.3.1(f)	**A*: use a lower-bound heuristic**	79-82
2.3.1(a-f)	**Synoptic algorithm decisions**	83-86

NAVIGATION

2.3 / INDEPENDENT REVISION

How to use each four-page cycle

A predictable routine for learners reading independently, revisiting lessons or preparing for assessments.

01 UNDERSTAND

Read the explanation, follow the visual and recall highlighted vocabulary.

02 MODEL

Trace the worked example one change at a time; explain why each step happens.

03 TRY

Answer in black pen before viewing the next page. Show intermediate workings.

04 CHECK

Self-mark in green; correct one misconception and explain the improvement.

school house style before publication.

2.3 / ALGORITHMS

Understand it. Model it. Try it. Check it.

This chapter spans OCR H446 2.3.1(a-f) and connects algorithm design, data structures, complexity and standard algorithms.

2.3.1(a,b)

Design and suitability

Requirements, constraints, correct choices.

2.3.1(c,d)

Efficiency

Constant, logarithmic, polynomial and exponential growth.

2.3.1(e)

Data structures

Stacks, queues, linked lists, trees, DFS and BFS.

2.3.1(f)

Standard algorithms

Linear/binary search; four sorts; Dijkstra and A*.

Try each exam question before using the separate indicative answers.

2.3 / SPECIFICATION CHECKLIST

Check every OCR requirement

The OCR items are summarised here. Page ranges are for this chapter PDF; tick only after applying each idea.

2.3.1(a)	Analyse/design algorithms	7-14
2.3.1(b)	Choose for data, time and space	11-18
2.3.1(c)	Efficiency, Big O and measurement	15-26
2.3.1(d)	Compare complexity classes	19-26
2.3.1(e)	Stacks and queues	27-34
2.3.1(e)	Linked lists and trees	35-42
2.3.1(e)	Post-order DFS and BFS traversal	43-50
2.3.1(f)	Linear and binary search	51-58
2.3.1(f)	Bubble and insertion sort	59-66
2.3.1(f)	Merge and quick sort	67-74
2.3.1(f)	Dijkstra and A*	75-82
2.3.1(a-f)	Synoptic selection/justification	83-86

OFFICIAL SOURCE

Use the examples to practise applying each specification requirement.

2.3.1(A) / SEE IT / CONCEPT AND VISUAL

Design an algorithm from a brief

An algorithm is a finite, unambiguous sequence of steps. Begin by deciding what enters, what leaves and which cases can break the proposed solution.

INPUT

Data supplied, its type and permitted range.

OUTPUT

The required result and its format.

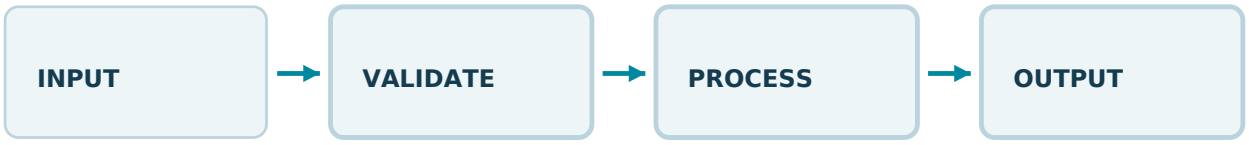
RULE

Precisely which transformation is needed.

TEST

Normal, boundary and invalid cases.

HOW THE PARTS FIT TOGETHER



INPUT -> VALIDATE -> PROCESS -> OUTPUT

VISUAL MEMORY BRIDGE

Think of a recipe: ingredients are inputs, the method is the algorithm, and the finished dish is the output. A good recipe also says what to do when an ingredient is missing.

WORDS THAT EARN MARKS

Algorithm	Finite steps	**Precondition**	Required before
Postcondition	Required after	**Boundary**	Edge value

EXAM TRAP

Do not jump from a scenario straight to code; unspecified inputs and failure cases become hidden bugs.

2.3.1(A) / WORKED EXAMPLE / STEP BY STEP

Design a delivery-charge algorithm

Orders of at least GBP 40 have free delivery; smaller orders pay GBP 4.50. Negative prices are invalid.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Input	Read subtotal (number)	Identify the single needed input.
2 Validate	If subtotal < 0, reject	An invalid value must not be charged.
3 Branch	If subtotal >= 40 then delivery = 0	The boundary is inclusive.
4 Else	Otherwise delivery = 4.50	Handle valid orders below the limit.
5 Calculate	total = subtotal + delivery	Return the complete bill.
6 Test	39.99, 40.00, -1	Check both sides and invalid input.

EXPLAIN THE METHOD

State the inclusive threshold explicitly and make error handling part of the design.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(A) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Identify the input, output and one precondition for the delivery-charge algorithm.

QUESTION 2 / 5 MARKS

Trace the charge and final total for subtotals 39.99 and 40.00. Explain the difference.

QUESTION 3 / 6 MARKS

Write pseudocode that validates subtotal, chooses the delivery charge and outputs either INVALID or the final total.

2.3.1(A) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Input: order subtotal [1].
- Output: total payable or an error indication [1].
- Precondition: subtotal is a numeric value, or otherwise is validated [1].

Q2 / 5 MARKS

- At 39.99, delivery = 4.50 [1], total = 44.49 [1].
- At 40.00, delivery = 0 [1], total = 40.00 [1].
- The condition uses >=, including exactly 40.00 [1].

Q3 / 6 MARKS

- INPUT subtotal [1].
- IF subtotal < 0 THEN [1].
- OUTPUT "INVALID" [1].
- ELSE determine whether subtotal >= 40 [1].
- Select delivery 0 or 4.50 as appropriate [1].
- OUTPUT subtotal + delivery [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Check that the output and boundary test are both explicit.

2.3.1(A,B) / SEE IT / CONCEPT AND VISUAL

Choose an algorithm for the situation

The best algorithm depends on the task, input properties and resource limits. Correctness comes first; time and memory trade-offs then determine suitability.

CORRECT?

Produces the required answer for all allowed cases.

INPUT

Size, ordering and update frequency.

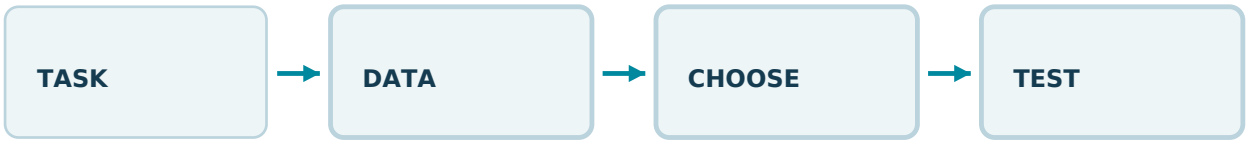
TIME

Work done as input size increases.

SPACE

Extra storage required.

HOW THE PARTS FIT TOGETHER



SORTED DATA? YES -> BINARY SEARCH NO -> LINEAR SEARCH

VISUAL MEMORY BRIDGE

Choose a route based on road conditions: a motorway is fast only when it is available. Likewise binary search is fast only when the data is already ordered.

WORDS THAT EARN MARKS

Constraint	Limiting condition	**Sorted**	Ordered data
Trade-off	One cost vs another	**Correctness**	Always right

EXAM TRAP

Do not call binary search the best search for an unsorted list unless sorting cost is justified.

2.3.1(A,B) / WORKED EXAMPLE / STEP BY STEP

Pick a search for a one-off lookup

A class list has 30 unsorted names and a teacher wants to check one name once.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Inspect	30 names; not sorted	Record the actual data condition.
2 Option A	Linear search: scan until found	Works without preprocessing.
3 Option B	Sort then binary search	Adds a sort for only one query.
4 Compare	One small search vs sort + search	Include setup work.
5 Select	Linear search	Simpler and appropriate here.
6 Reconsider	For many future searches, maintain sorted order	Changed constraints can change the choice.

EXPLAIN THE METHOD

Suitability is a justified decision, not a memorised ranking.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(A,B) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State three factors affecting the choice between searching algorithms.

QUESTION 2 / 5 MARKS

Explain why linear search may be preferable for a single search of 30 unsorted names.

QUESTION 3 / 6 MARKS

A million records are kept sorted and searched thousands of times each day. Recommend a search and justify it against an alternative, including a limitation.

2.3.1(A,B) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Whether the data is sorted [1].
- Number of items or data size [1].
- Number of searches / time or memory constraints [1].

Q2 / 5 MARKS

- Linear search can run on unsorted data [1].
- It needs no sorting step [1].
- The list is small [1].
- There is only one search [1].
- Sorting first would incur extra time / complexity [1].

Q3 / 6 MARKS

- Binary search is suitable because records are sorted [1].
- It repeatedly halves the remaining range [1].
- Time is O(log n) per lookup [1].
- Thousands of repeated lookups make this valuable [1].
- Linear search may require O(n) comparisons [1].
- Maintaining sort order on insertions can be costly / binary search requires random access [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Separate the cost of preparation from the cost of a single search.

2.3.1(B,C) / SEE IT / CONCEPT AND VISUAL

Analyse time and space costs

Time complexity counts how work grows with input size n; auxiliary space measures additional memory used beyond the input. Execution timings can help but depend on hardware.

INPUT SIZE

Define n before counting operations.

TIME

Count dominant operations or passes.

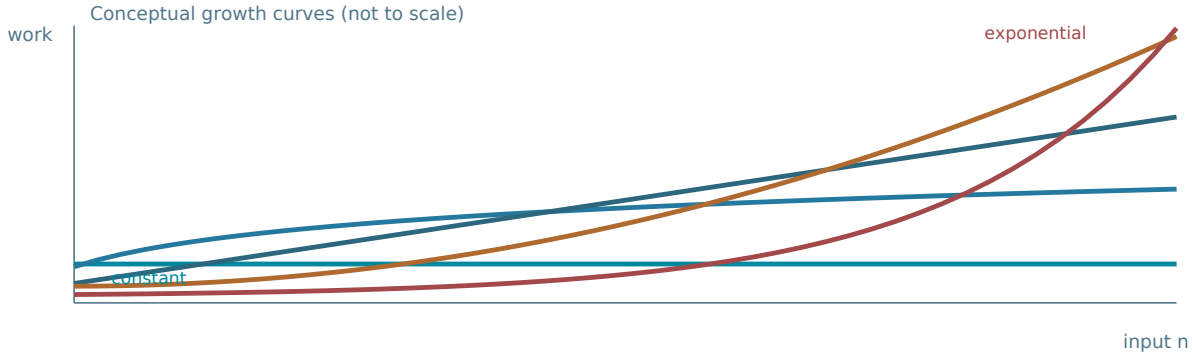
SPACE

Count extra arrays, stack frames or buffers.

EMPIRICAL

Measure repeated runs with fair conditions.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

Packing a suitcase: time is how many items you inspect; extra space is how many spare bags you need. Counting them separately prevents misleading comparisons.

WORDS THAT EARN MARKS

n	Input size	**Time**	Growth of work
Auxiliary	Extra storage	**Worst case**	Maximum work

EXAM TRAP

O(1) space does not mean zero memory; it means extra storage does not grow with n.

2.3.1(B,C) / WORKED EXAMPLE / STEP BY STEP

Count a nested loop

FOR i = 1 TO n: FOR j = 1 TO n: comparisons = comparisons + 1.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Define	n = loop bound	The same n controls both loops.
2 Inner	n iterations per outer pass	For each i, j runs n times.
3 Outer	n complete passes	There are n values of i.
4 Count	n x n = n^2 increments	Multiply, do not add.
5 Time	O(n^2)	The dominant term is quadratic.
6 Space	O(1) auxiliary	Only the loop variables and counter are stored.

EXPLAIN THE METHOD

Use a trace to explain the count; then express its growth using Big O.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(B,C) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define input size, time complexity and auxiliary space.

QUESTION 2 / 5 MARKS

For n = 4, count the inner-loop increments in the given nested loop and explain the general expression.

QUESTION 3 / 6 MARKS

An algorithm copies all n items to a new array, then scans the array once. Determine its time and auxiliary space complexities with reasons.

2.3.1(B,C) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- n is a measurable amount of input [1].
- Time complexity describes growth of required operations [1].
- Auxiliary space describes additional memory used as input grows [1].

Q2 / 5 MARKS

- Outer loop runs four times [1].
- Inner loop runs four times each pass [1].
- There are 16 increments [1].
- In general n x n = n^2 [1].
- Time complexity is O(n^2) [1].

Q3 / 6 MARKS

- Copy loop processes each item once, O(n) [1].
- Scan processes each item once, O(n) [1].
- Together 2n grows as O(n) [1].
- New array holds n values [1].
- Auxiliary space is O(n) [1].
- Sequential loops add rather than multiply [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. When counting loops, distinguish nested loops from consecutive loops.

2.3.1(C,D) / SEE IT / CONCEPT AND VISUAL

Recognise constant, log and linear

Big O describes an upper bound on asymptotic growth. Constant time does not grow with n; logarithmic growth rises slowly; linear work grows proportionally with n.

O(1)

A fixed number of operations.

O(log n)

Repeatedly shrink the problem.

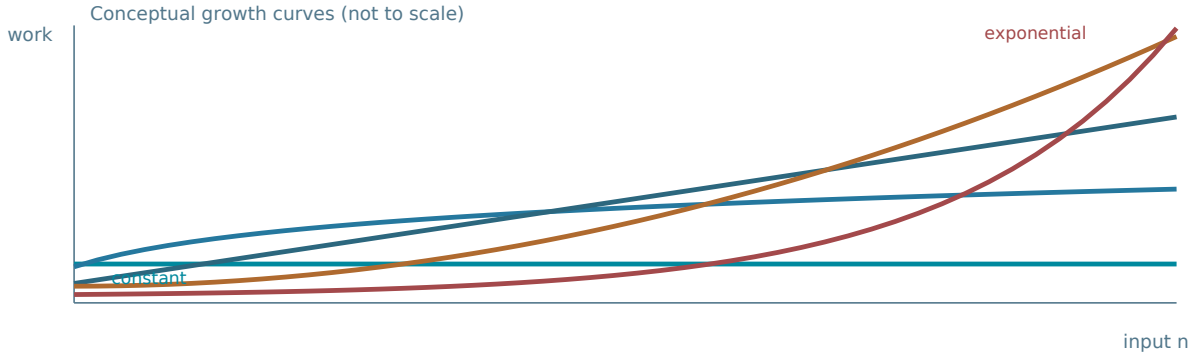
O(n)

Visit a number of items proportional to n.

ASSUMPTION

Know whether data is indexed or sorted.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

A library: take a labelled book from a known shelf slot (constant); use a sorted catalogue by halving ranges (logarithmic); check every spine (linear).

WORDS THAT EARN MARKS

O(1)	Constant	**O(log n)**	Logarithmic
O(n)	Linear	**Asymptotic**	As n grows

EXAM TRAP

Do not infer O(log n) from a short run: the algorithm must actually reduce the candidate set each step.

2.3.1(C,D) / WORKED EXAMPLE / STEP BY STEP

Compare work for 1,024 items

Assume a list provides direct indexed access and a binary-search list is sorted.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Constant	Access index 37: one lookup	Position is already known.
2 Linear	Worst-case scan: 1,024 checks	Last item may be the target.
3 Binary	Halve 1,024 -> 512 -> ... -> 1	Ten halvings reduce 1,024 to one.
4 Classify	O(1), O(n), O(log n)	Classify growth, not milliseconds.
5 Preconditions	Binary search requires sorted data	Otherwise the discard decision fails.
6 Distinguish	Worst-case count depends on algorithm	All three can be correct for different tasks.

EXPLAIN THE METHOD

Being faster for one measured n is not the definition of a better Big O class.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(C,D) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Match indexed access, linear search and binary search (sorted list) to their Big O time classes.

QUESTION 2 / 5 MARKS

Explain why binary search uses roughly 10 halvings on 1,024 ordered items and why this is logarithmic.

QUESTION 3 / 6 MARKS

Compare linear and binary search for a 1,024-item list when data is unsorted and used once; include the sorting prerequisite and overall work.

2.3.1(C,D) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Indexed access is O(1) [1].
- Linear search worst case is O(n) [1].
- Binary search worst case is O(log n) [1].

Q2 / 5 MARKS

- 1,024 = 2^10 [1].
- Each comparison removes about half the candidates [1].
- Ten halvings leave one candidate [1].
- Doubling n adds only about one extra halving [1].
- This growth is O(log n) [1].

Q3 / 6 MARKS

- Binary search requires the list to be sorted [1].
- An unsorted list must first be sorted or maintained sorted [1].
- This setup has a cost [1].
- Linear search works immediately on unsorted data [1].
- Worst-case linear work is O(n) [1].
- For one lookup, sorting plus binary search may not be worth it [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Explain why halving gives logarithmic growth, not just the label.

2.3.1(C,D) / SEE IT / CONCEPT AND VISUAL

Compare polynomial and exponential

Quadratic work is polynomial; doubling choices at every step may create exponential work. For large n an exponential algorithm can become infeasible.

O(n^2)

A common nested-loop growth rate.

O(n^3)

Polynomial, but steeper than quadratic.

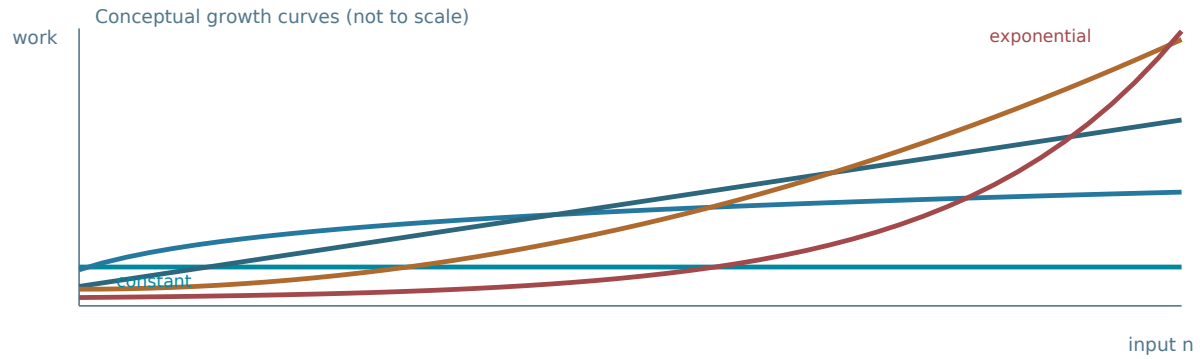
O(2^n)

Exponential doubling with each extra input.

COMPARE

Ignore constants for asymptotic class.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

A seating chart: comparing every person with every person grows roughly as a square; considering every yes/no invitation pattern doubles the possibilities with each guest.

WORDS THAT EARN MARKS

Quadratic	n squared	**Polynomial**	n to fixed power
Exponential	Base to n	**Scalability**	Larger n

EXAM TRAP

O(2^n) is not the same as O(n^2); compare what is the exponent and what is the base.

2.3.1(C,D) / WORKED EXAMPLE / STEP BY STEP

Estimate growth before implementation

Two approaches require n^2 operations and 2^n operations respectively.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Input	n = 8	Select a comparison size.
2 Quadratic	8^2 = 64	A polynomial number of operations.
3 Exponential	2^8 = 256	All binary choices across eight inputs.
4 Increase	n = 12	Four more input items.
5 Quadratic	12^2 = 144	A moderate increase.
6 Exponential	2^12 = 4,096	A 16-fold increase for four more items.

EXPLAIN THE METHOD

Complexity classes describe scaling; real timing also depends on constants and implementation.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(C,D) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Classify O(n^2), O(n^3) and O(2^n) as polynomial or exponential.

QUESTION 2 / 5 MARKS

Calculate n^2 and 2^n for n = 5 and n = 10, and state which grows faster.

QUESTION 3 / 6 MARKS

A brute-force algorithm tests every subset of n choices. Explain why its worst-case work is exponential and suggest one strategy for reducing practical search.

2.3.1(C,D) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- O(n^2) is polynomial [1].
- O(n^3) is polynomial [1].
- O(2^n) is exponential [1].

Q2 / 5 MARKS

- At n=5: n^2 = 25 [1], 2^n = 32 [1].
- At n=10: n^2 = 100 [1], 2^n = 1,024 [1].
- Exponential grows faster for large n [1].

Q3 / 6 MARKS

- Each input can be included or excluded [1].
- There are 2^n subsets [1].
- Naively trying each subset costs at least O(2^n) [1].
- This scales quickly as n increases [1].
- Pruning/backtracking may discard impossible branches [1].
- Memoisation or another problem-specific algorithm can avoid repeated work [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. State that polynomial and exponential refer to functions of n.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Trace a stack: LIFO

A stack inserts and removes items at one end called the top. PUSH and POP are normally O(1) using a linked stack or dynamic array with amortised append.

PUSH(x)

Place x on top.

POP()

Remove and return top.

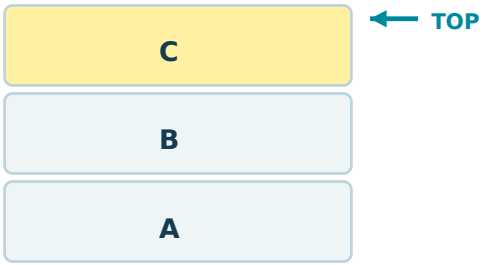
PEEK()

Read top without removing.

EMPTY

Guard against stack underflow.

HOW THE PARTS FIT TOGETHER



PUSH / POP at the TOP • Last in, first out

VISUAL MEMORY BRIDGE

Like a stack of trays: take the latest tray placed at the top before those underneath.

WORDS THAT EARN MARKS

Stack	LIFO	**Push**	Insert top
Pop	Remove top	**Underflow**	Remove from empty

EXAM TRAP

POP returns the most recently pushed item still on the stack, not the earliest.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Simulate six stack actions

Start with an empty stack; represent contents bottom-to-top.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 PUSH A	[A]	A is the first item and the bottom.
2 PUSH B	[A, B]	B becomes the top.
3 PUSH C	[A, B, C]	C becomes the top.
4 POP	returns C; [A, B]	LIFO removes newest.
5 PUSH D	[A, B, D]	D is now at top.
6 PEEK	returns D; unchanged	Peek does not remove the item.

EXPLAIN THE METHOD

Draw the stack after every operation; record both returned values and remaining contents.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define LIFO and the operations PUSH and POP.

QUESTION 2 / 5 MARKS

Starting empty, trace PUSH 4, PUSH 7, POP, PUSH 9, POP. Give returned values and final contents.

QUESTION 3 / 6 MARKS

Write pseudocode for safe POP on an array-backed stack with topIndex = -1 when empty; explain what happens if it is empty.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- LIFO means last in, first out [1].
- PUSH inserts at the top [1].
- POP removes/returns the top [1].

Q2 / 5 MARKS

- PUSH 4 -> stack contains 4 [1].
- PUSH 7 -> [4,7] [1].
- First POP returns 7 [1].
- PUSH 9 -> [4,9] [1].
- Second POP returns 9; only 4 remains [1].

Q3 / 6 MARKS

- IF topIndex = -1 THEN [1].
- Signal EMPTY/UNDERFLOW without removing [1].
- ELSE read stack[topIndex] [1].
- Decrease topIndex by 1 [1].
- Return stored item [1].
- Explain that attempting POP when empty must not access an invalid index [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep TOP and returned items separate in the trace.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Trace a queue: FIFO

A queue inserts at the rear and removes at the front. ENQUEUE and DEQUEUE support first-in-first-out service; a circular array avoids shifting every item.

ENQUEUE(x)

Insert at rear.

DEQUEUE()

Remove from front.

FRONT

Item waiting the longest.

CIRCULAR

Wrap array indices when needed.

HOW THE PARTS FIT TOGETHER



DEQUEUE at FRONT • ENQUEUE at REAR

VISUAL MEMORY BRIDGE

Like a ticket queue: the person who joined first is served first, unless a separate priority rule is introduced.

WORDS THAT EARN MARKS

Queue	FIFO	**Enqueue**	Add at rear
Dequeue	Remove front	**Overflow**	Queue full

EXAM TRAP

Do not implement a fixed-array dequeue by always shifting elements if O(1) operations are required; use head/tail pointers.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Simulate a circular queue of capacity four

Empty queue, front=0, rear=0; rear is the next free slot.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 ENQ A	[A]; rear=1	Write at index 0, then advance.
2 ENQ B	[A,B]; rear=2	B goes behind A.
3 DEQ	returns A; front=1	Remove oldest item.
4 ENQ C	[B,C]; rear=3	C follows B in logical order.
5 ENQ D	[B,C,D]; rear=0	Index wraps modulo four.
6 DEQ	returns B; front=2	Only logical front moves.

EXPLAIN THE METHOD

Track front, rear and number of items; wrapping an index does not rearrange logical order.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define FIFO and distinguish ENQUEUE from DEQUEUE.

QUESTION 2 / 5 MARKS

Starting empty, trace ENQUEUE 3, ENQUEUE 6, DEQUEUE, ENQUEUE 8, DEQUEUE. Give outputs and remaining data.

QUESTION 3 / 6 MARKS

Explain how a circular queue with array capacity 5 can provide O(1) enqueue/dequeue and identify two error conditions.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- FIFO means first in, first out [1].
- ENQUEUE inserts at the rear [1].
- DEQUEUE removes from the front [1].

Q2 / 5 MARKS

- After first two adds [3,6] [1].
- First DEQUEUE returns 3 [1].
- ENQUEUE 8 leaves [6,8] [1].
- Second DEQUEUE returns 6 [1].
- Only 8 remains in the queue [1].

Q3 / 6 MARKS

- Maintain front pointer [1] and rear pointer [1].
- Update indices modulo capacity [1].
- No O(n) shifting is required, so each operation can be O(1) [1].
- Underflow if queue empty [1].
- Overflow if array queue full [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Follow the logical order independently of physical array indices.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Follow a linked list

A linked-list node stores data and a reference to the next node. Traversal follows links; inserting at a known head can be O(1), while finding a value is O(n).

NODE

Data and next pointer.

HEAD

Reference to first node.

TRAVERSE

Follow next until null.

INSERT

Update links without losing nodes.

HOW THE PARTS FIT TOGETHER



Each arrow stores the location of the next node, not a fixed index.

VISUAL MEMORY BRIDGE

A treasure hunt: each clue gives the next location. You cannot jump to clue 12 without following the chain.

WORDS THAT EARN MARKS

Node	Value plus link	**Head**	First node
Null	No successor	**Pointer**	Reference

EXAM TRAP

When inserting between nodes, preserve the old successor before changing the links.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Insert 7 between linked nodes 4 and 9

Initially HEAD -> 4 -> 9 -> 12 -> NULL; current points at the node containing 4.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Allocate	new.data = 7	Create the new node.
2 Preserve	new.next = current.next	The new node points to 9.
3 Relink	current.next = new	The node 4 now points to 7.
4 Trace	4 -> 7 -> 9 -> 12	No old node is lost.
5 Find 12	Check 4,7,9,12	Following links is linear.
6 Complexity	Known-position insert O(1)	Searching for the position can be O(n).

EXPLAIN THE METHOD

A linked list supports cheap pointer updates but lacks O(1) indexing by position.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State the components of a singly linked-list node and the purpose of HEAD.

QUESTION 2 / 5 MARKS

Show link changes needed to insert a node 7 after node 4 in 4 -> 9 -> NULL.

QUESTION 3 / 6 MARKS

Compare inserting at the head of a linked list with accessing the kth item of an array or linked list in time-complexity terms.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Each node contains data [1].
- Each node contains a next reference [1].
- HEAD identifies the first node [1].

Q2 / 5 MARKS

- Create a new node containing 7 [1].
- Set new.next to node 9 [1].
- Set node 4 next to new [1].
- Final chain 4 -> 7 -> 9 -> NULL [1].
- Old successor must not be lost [1].

Q3 / 6 MARKS

- Head insertion: set new.next=head [1], head=new [1].
- Known-head insertion is O(1) [1].
- Array random indexed access is O(1) [1].
- Linked-list kth-item access follows k links, O(n) worst case [1].
- Trade-off: pointers use extra storage / arrays may require shifting on insert [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Show where each next reference points after the operation.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Search a binary search tree

In a binary search tree (BST), values smaller than a node go left and larger values go right (with a declared duplicate policy). Height determines search performance.

ROOT

Top of the tree.

LEFT

Smaller subtree for a BST.

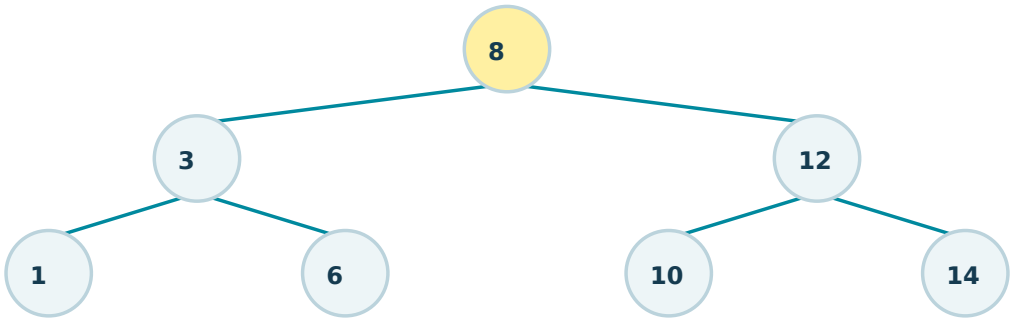
RIGHT

Larger subtree for a BST.

HEIGHT

Longest root-to-leaf distance.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

Like a choose-your-own-path signpost: every comparison tells you which branch cannot hold the value.

WORDS THAT EARN MARKS

Root	Top node	**Leaf**	No children
BST	Ordered branches	**Balanced**	Shallow height

EXAM TRAP

BST search is O(log n) only when the tree height is logarithmic; an unbalanced chain can give O(n).

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Locate 10 in a binary search tree

Tree root 8, left child 3, right child 12; 12 has children 10 and 14.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Root	Compare 10 with 8	10 > 8, so choose right.
2 Right	Visit 12	10 < 12, so choose left.
3 Left	Visit 10	Value matches.
4 Path	8 -> 12 -> 10	Three node visits.
5 Balanced	Height grows slowly	Expected O(log n) on balanced BST.
6 Skewed	Chain-like tree	Worst-case search may be O(n).

EXPLAIN THE METHOD

Traversal order and BST search are different algorithms: one explores, the other uses ordering to prune.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define root, leaf and binary search tree.

QUESTION 2 / 5 MARKS

Trace the nodes visited when searching for 1 and then 14 in the tree described above.

QUESTION 3 / 6 MARKS

Explain why inserting sorted values into a plain BST can cause poor search performance, and give one way to avoid it.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Root is the entry/top node [1].
- Leaf has no children [1].
- BST places smaller values left and larger right (according to a duplicate rule) [1].

Q2 / 5 MARKS

- Search 1: 8 -> 3 -> 1 [2].
- Search 14: 8 -> 12 -> 14 [2].
- At each step comparison chooses one branch [1].

Q3 / 6 MARKS

- Sorted insertion may create a one-sided chain [1].
- Height can become O(n) [1].
- Search then visits up to n nodes [1].
- No halving is guaranteed [1].
- Use a self-balancing tree or rebuild to balance [1].
- A balanced tree keeps search roughly O(log n) [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Label the direction chosen at each comparison.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Depth-first post-order traversal

Post-order depth-first traversal visits every left subtree, then every right subtree, then the parent: LEFT, RIGHT, ROOT. It is often used when children must be processed first.

DEPTH FIRST

Explore a branch before siblings.

POST-ORDER

Left, right, root.

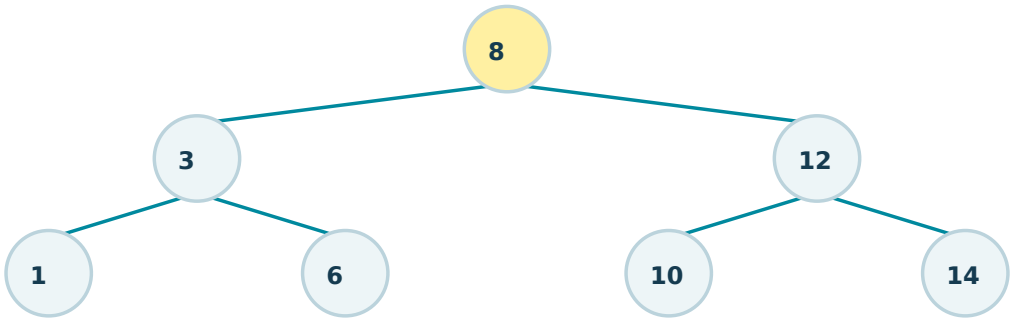
STACK

Call stack remembers unfinished nodes.

TREE

Visit each node once.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

Pack up a treehouse: clear each child room before you dismantle its parent platform.

WORDS THAT EARN MARKS

DFS	Depth-first search	**Post-order**	L-R-Root
Recursion	Self-calling	**Stack**	Pending work

EXAM TRAP

In post-order the root is last; preorder (root first) and inorder (root between) are different.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Produce the post-order list

Tree: root 8; children 3 and 12; 3 has children 1 and 6; 12 has children 10 and 14.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Left-left	Visit 1	Leaf has no subtrees.
2 Left-right	Visit 6	Second leaf under 3.
3 Left-root	Visit 3	Both children of 3 complete.
4 Right-left	Visit 10	First leaf under 12.
5 Right-right/root	Visit 14 then 12	Finish right subtree.
6 Root	Visit 8	Output: 1,6,3,10,14,12,8.

EXPLAIN THE METHOD

Write the root only after both of its subtrees have been fully traversed.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State the order of operations in recursive post-order traversal.

QUESTION 2 / 5 MARKS

Give the full post-order traversal for root A, with B and C as children; B has D,E and C has F.

QUESTION 3 / 6 MARKS

Write recursive pseudocode for post-order traversal and explain its time complexity for n nodes.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Traverse left subtree first [1].
- Traverse right subtree second [1].
- Process root last [1].

Q2 / 5 MARKS

- D then E [1].
- B follows its children [1].
- F then C [1].
- A is last [1].
- Full sequence D,E,B,F,C,A [1].

Q3 / 6 MARKS

- PROCEDURE post(node) [1].
- IF node != NULL THEN [1].
- post(node.left) [1].
- post(node.right) [1].
- OUTPUT node.value [1].
- Every node is visited once, so O(n) time [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Use the L-R-Root rule locally at each subtree, not only at the main root.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Breadth-first traversal of a tree

Breadth-first traversal explores a tree level by level and uses a FIFO queue to remember discovered nodes. Do not confuse it with post-order DFS.

BFS

Explore current depth before next.

QUEUE

Hold discovered nodes.

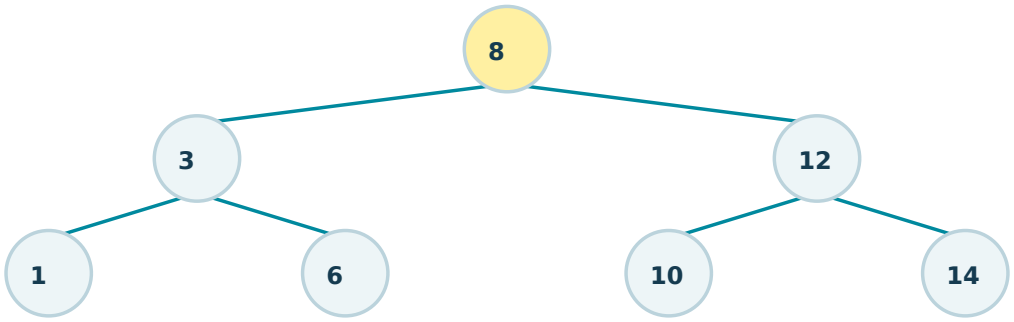
ENQUEUE

Add each child left-to-right.

VISIT

Process a node when removed.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

A theatre fills one row at a time rather than following one aisle all the way to the back before returning.

WORDS THAT EARN MARKS

Breadth first	Level order	**FIFO**	Queue policy
Enqueue	Add rear	**Dequeue**	Remove front

EXAM TRAP

Using a stack instead of a queue usually changes the visit order into depth-first behaviour.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Trace BFS with the queue visible

Tree: root 8; children 3,12; 3 has 1,6; 12 has 10,14. Queue is shown front-to-rear.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Start	Queue [8]	Add root.
2 Visit 8	Queue [3,12]	Enqueue its children.
3 Visit 3	Queue [12,1,6]	Add 1 and 6 after 12.
4 Visit 12	Queue [1,6,10,14]	Add 10 and 14.
5 Visit leaves	1,6,10,14	No children to add.
6 Output	8,3,12,1,6,10,14	Nodes appear in depth order.

EXPLAIN THE METHOD

When tracing, record both the output sequence and the queue contents after each removal.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Name the supporting data structure and the visitation pattern used in breadth-first tree traversal.

QUESTION 2 / 5 MARKS

For tree root A, children B,C, and B has D,E, write the BFS visit order and the queue after visiting A.

QUESTION 3 / 6 MARKS

Write an outline of breadth-first tree traversal with a queue; justify O(n) time for n nodes.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- BFS uses a queue [1].
- Queue is first in, first out [1].
- BFS processes one level before the next [1].

Q2 / 5 MARKS

- Visit order A,B,C,D,E [3].
- Queue after visiting A is [B,C] [2].

Q3 / 6 MARKS

- Add root to queue [1].
- WHILE queue not empty [1].
- Remove front node [1].
- Process/output that node [1].
- Enqueue its children [1].
- Each node is enqueued and removed once, O(n) time [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. After adding children, make sure earlier queued nodes stay at the front.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Linear search

Linear search checks elements one by one until a match is found or the list ends. It works for unsorted data and has O(n) worst-case time.

COMPARE

Test current item against target.

MATCH

Return position or found state.

NOT FOUND

Terminate after final item.

COMPLEXITY

Worst case O(n), extra space O(1).

HOW THE PARTS FIT TOGETHER



ARRAY [8, 3, 12, 5, 7] ; target 5 ; check 8,3,12,5

VISUAL MEMORY BRIDGE

Look for a lost pen by checking every desk, stopping as soon as you find it.

WORDS THAT EARN MARKS

Target	Search key	**Index**	Position
Sentinel	Not-found value	**Worst case**	Last or absent

EXAM TRAP

Avoid off-by-one errors: position 0 is the first element when using zero-based indexing.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Trace a zero-based linear search

items = [8,3,12,5,7] and target = 5.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 i=0	items[0] = 8: no	Continue.
2 i=1	items[1] = 3: no	Continue.
3 i=2	items[2] = 12: no	Continue.
4 i=3	items[3] = 5: yes	Match found.
5 Return	index 3	Do not return 4 for zero-based data.
6 Bound	Worst-case 5 comparisons	Missing target or last item.

EXPLAIN THE METHOD

State whether your result is an index or a human-friendly ordinal position.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State a benefit, worst-case time complexity and extra-space complexity of linear search.

QUESTION 2 / 5 MARKS

Trace comparisons when finding 5 in [8,3,12,5,7] and state the zero-based index returned.

QUESTION 3 / 6 MARKS

Write pseudocode to search an unsorted array of length n and return -1 when no match exists.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Works with unsorted data [1].
- Worst-case O(n) [1].
- Auxiliary space O(1) [1].

Q2 / 5 MARKS

- Checks 8 [1], 3 [1], 12 [1], 5 [1].
- Returns zero-based index 3 [1].

Q3 / 6 MARKS

- FOR i = 0 TO n-1 [1].
- IF arr[i] = target [1].
- RETURN i [1].
- End IF / loop correctly [1].
- RETURN -1 after loop [1].
- Explain -1 means not found, not a valid index [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. When not found, scan all valid indices before returning -1.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Binary search

Binary search repeatedly compares with the middle of an ordered range, discarding half. For a sorted indexable array, its worst-case time is O(log n).

SORTED

Required input property.

LOW / HIGH

Inclusive candidate bounds.

MID

(low + high) DIV 2.

HALVE

Discard the impossible half.

HOW THE PARTS FIT TOGETHER



SORTED [2,5,8,11,14,17,20]; target 17

VISUAL MEMORY BRIDGE

Find a word in a dictionary by opening near the middle and deciding which half cannot contain it.

WORDS THAT EARN MARKS

Sorted	Ordered	**Midpoint**	Middle index
DIV	Integer quotient	**Halving**	Logarithmic

EXAM TRAP

Do not apply binary search to unsorted data; also update low = mid+1 or high = mid-1 to avoid repeating a midpoint.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Trace a successful binary search

arr = [2,5,8,11,14,17,20], target = 17, indices 0..6.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Start	low=0, high=6	All seven items are candidates.
2 Mid	mid=(0+6) DIV 2=3	arr[3]=11.
3 Discard	17>11, low=4	Discard indices 0..3.
4 Mid	mid=(4+6) DIV 2=5	arr[5]=17.
5 Found	Return index 5	Two comparisons in this trace.
6 Absent	Terminate when low>high	There is no candidate left.

EXPLAIN THE METHOD

After each comparison, cross out the impossible half and update the inclusive bounds.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State a prerequisite for binary search, its worst-case Big O, and what causes it to terminate unsuccessfully.

QUESTION 2 / 5 MARKS

Trace binary search for 5 in [2,5,8,11,14,17,20]. Show mid indices and bound updates.

QUESTION 3 / 6 MARKS

Write iterative pseudocode for binary search using low, high and mid, returning -1 for absent values.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Array must be sorted [1].
- Worst-case O(log n) [1].
- Unsuccessful when low > high [1].

Q2 / 5 MARKS

- Start low=0, high=6, mid=3 with value 11 [1].
- 5 < 11 so high=2 [1].
- Next mid=1, value 5 [1].
- Match at index 1 [1].
- The discarded half cannot contain the value in sorted order [1].

Q3 / 6 MARKS

- Set low=0 and high=n-1 [1].
- WHILE low <= high [1].
- mid = (low+high) DIV 2 [1].
- If equal return mid [1].
- If target > arr[mid], low=mid+1; else high=mid-1 [1].
- RETURN -1 after loop [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Record low, high and mid on every iteration.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Bubble sort

Bubble sort compares neighbours, swapping them when out of order. Each completed pass moves a maximum to the right in ascending order.

COMPARE

Adjacent elements.

SWAP

Only if left > right for ascending.

PASS

Largest unsorted value moves right.

STOP

After no swaps, or all passes.

HOW THE PARTS FIT TOGETHER



PASS 1: [5,1,4,2] -> [1,4,2,5]

VISUAL MEMORY BRIDGE

Like bubbles rising: the largest value drifts to its final position at the end of each pass.

WORDS THAT EARN MARKS

Adjacent	Neighbours	**Swap**	Exchange
Pass	Sweep	**Stable**	Equal relative order

EXAM TRAP

Do not claim a full sort after one pass; only the largest remaining element is guaranteed to be placed.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Trace the first two bubble passes

Sort [5,1,4,2] in ascending order, swapping only on >.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Compare 5,1	[1,5,4,2]	Swap because 5>1.
2 Compare 5,4	[1,4,5,2]	Swap because 5>4.
3 Compare 5,2	[1,4,2,5]	5 reaches last position.
4 Compare 1,4	unchanged	Already ordered.
5 Compare 4,2	[1,2,4,5]	Swap; sorted result.
6 Bounds	Worst O(n^2), best O(n) with early exit	Nested comparisons; no-swap pass stops.

EXPLAIN THE METHOD

Show the array state after swaps; count passes separately from individual comparisons.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Describe an ascending bubble-sort comparison, swap rule and outer-pass purpose.

QUESTION 2 / 5 MARKS

Show the array [4,3,2,1] after the first complete bubble-sort pass and state how many swaps occur.

QUESTION 3 / 6 MARKS

Compare bubble sort and insertion sort for nearly sorted data, considering early-exit optimisations and worst-case complexity.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Compare adjacent items [1].
- Swap if left item is greater [1].
- Each pass positions a largest unsorted value at right [1].

Q2 / 5 MARKS

- 4,3 swaps -> [3,4,2,1] [1].
- 4,2 swaps -> [3,2,4,1] [1].
- 4,1 swaps -> [3,2,1,4] [1].
- Three swaps [1].
- Largest item 4 is final [1].

Q3 / 6 MARKS

- Both are O(n^2) in worst case [1].
- Bubble with no-swap early exit can be O(n) best case [1].
- Insertion sort often uses fewer shifts on nearly sorted data [1].
- Insertion best case O(n) [1].
- Both can be stable in standard versions [1].
- State that actual choice depends on input order/implementation [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Write each swap, then check the rightmost fixed portion.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Insertion sort

Insertion sort grows a sorted prefix. Take the next key and shift larger prefix values one position right until the correct slot is free.

PREFIX

Items before key are sorted.

KEY

Next unsorted value.

SHIFT

Move larger values right.

INSERT

Place key in its correct slot.

HOW THE PARTS FIT TOGETHER



SORTED [2,5,8] | KEY 4 -> [2,4,5,8]

VISUAL MEMORY BRIDGE

Organise playing cards in your hand by inserting each new card into the already sorted set.

WORDS THAT EARN MARKS

Key	Current item	**Prefix**	Sorted left part
Shift	Move one place	**In-place**	Low extra space

EXAM TRAP

A shift is not necessarily a swap; retain the key while moving larger items.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Insert keys into a sorted prefix

Sort [5,2,4,1] ascending, showing the state after each key.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Start	[5 | 2,4,1]	First item forms a sorted prefix.
2 Key 2	[2,5 | 4,1]	Shift 5 then insert 2.
3 Key 4	[2,4,5 | 1]	Shift 5; place 4.
4 Key 1	[1,2,4,5]	Shift 5,4,2; insert 1.
5 Best case	Already sorted -> O(n)	Each key needs one comparison.
6 Worst case	Reverse order -> O(n^2)	Many shifts for each key.

EXPLAIN THE METHOD

Draw the boundary between sorted prefix and unsorted suffix after every outer iteration.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Explain the purpose of the key, sorted prefix and shift in insertion sort.

QUESTION 2 / 5 MARKS

Trace insertion sort on [3,1,2,4], giving the array after inserting each key.

QUESTION 3 / 6 MARKS

Explain why insertion sort can be efficient for nearly sorted arrays but O(n^2) for reverse-sorted arrays.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Key is the next item to insert [1].
- Prefix contains already sorted items [1].
- Shift creates space by moving larger prefix items [1].

Q2 / 5 MARKS

- Key 1 -> [1,3,2,4] [2].
- Key 2 -> [1,2,3,4] [2].
- Key 4 leaves list unchanged [1].

Q3 / 6 MARKS

- Nearly sorted: few shifts needed [1].
- Outer loop visits n-1 keys [1].
- Best-case comparisons grow O(n) [1].
- Reverse sorted: each key moves past many values [1].
- Sum 1+2+...+(n-1) is quadratic [1].
- Worst-case time O(n^2) [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep the key safe while shifting values; do not lose an element.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Merge sort

Merge sort splits the sequence into halves, recursively sorts each half and merges them into order. Standard array versions run in O(n log n) time with O(n) auxiliary storage.

DIVIDE

Split array into smaller halves.

RECURSE

Sort the two halves.

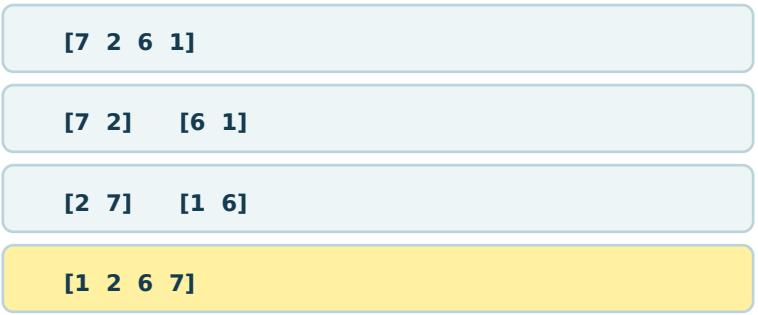
MERGE

Take the smaller next value.

SPACE

Temporary array for merging.

HOW THE PARTS FIT TOGETHER



The arrows show conceptual stages, not implementation-specific swaps.

VISUAL MEMORY BRIDGE

Two already sorted piles of exam papers can be combined by repeatedly taking the smallest paper from the two fronts.

WORDS THAT EARN MARKS

Divide and conque	Split and combine	**Merge**	Combine ordered
Stable	Preserve ties	**Auxiliary**	Extra array

EXAM TRAP

Merging works by comparing the fronts of sorted halves, not by concatenating the halves.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Divide and merge four values

Sort [7,2,6,1] in ascending order.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Split	[7,2] and [6,1]	Divide into equal halves.
2 Split again	[7],[2],[6],[1]	Single-element arrays are sorted.
3 Merge left	[2,7]	Compare 7 and 2.
4 Merge right	[1,6]	Compare 6 and 1.
5 Merge final	[1,2,6,7]	Repeatedly take smallest head.
6 Cost	O(n log n) time; O(n) array space	Log-depth splitting, linear merging per level.

EXPLAIN THE METHOD

At each merge, show both input halves already sorted.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State how merge sort splits, how it merges sorted halves, and its typical time complexity.

QUESTION 2 / 5 MARKS

Show all split levels and the merge result for [8,3,5,2].

QUESTION 3 / 6 MARKS

Explain why standard array merge sort takes O(n log n) time and uses O(n) extra space, and contrast this with an in-place sorting approach.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Divide/split [1].
- Recursively sort and merge/combine [1].
- Time O(n log n) [1].

Q2 / 5 MARKS

- Split [8,3] / [5,2] [1].
- Split to singletons (8), (3), (5), (2) [1].
- Merge to [3,8] [1] and [2,5] [1].
- Final [2,3,5,8] [1].

Q3 / 6 MARKS

- There are O(log n) split levels [1].
- Each level performs O(n) merging [1].
- Total O(n log n) [1].
- Temporary merge buffer holds O(n) values [1].
- An in-place method may need O(1) auxiliary storage [1].
- Trade-off: stable predictable time vs buffer space [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Never compare unsorted halves as if they were already merged.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Quick sort

Quick sort chooses a pivot and partitions values so smaller ones lie to one side, larger ones to the other. It recursively sorts each partition.

PIVOT

Chosen reference value.

PARTITION

Rearrange values around pivot.

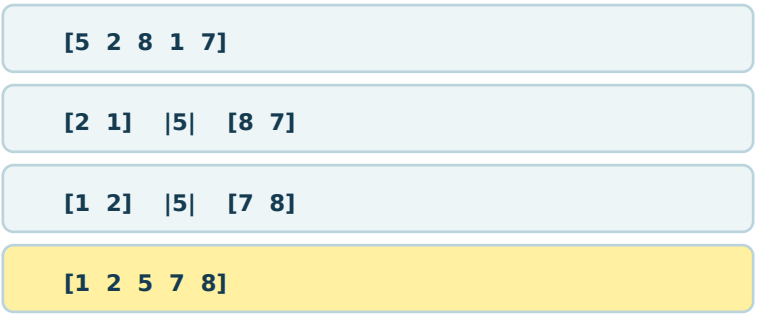
RECURSE

Sort both partitions.

PERFORMANCE

Average O(n log n); worst O(n^2).

HOW THE PARTS FIT TOGETHER



The arrows show conceptual stages, not implementation-specific swaps.

VISUAL MEMORY BRIDGE

Choose a team captain as a reference height; place shorter pupils left and taller pupils right, then organise each side.

WORDS THAT EARN MARKS

Pivot	Reference item	**Partition**	Split by pivot
Worst case	Unbalanced	**Recursion**	Repeated subproblem

EXAM TRAP

The exact array after one partition depends on the chosen pivot and partition scheme; do not assume a single trace for all implementations.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Partition around pivot 5

Illustrative partition (not a specific in-place swap scheme): input [5,2,8,1,7].

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Pivot	Choose 5	State your pivot rule.
2 Compare 2	2 < 5 -> left	Collect lower values.
3 Compare 8	8 > 5 -> right	Collect higher values.
4 Compare 1,7	1 left; 7 right	All items assigned.
5 Recur	Sort [2,1] and [8,7]	Solve independent partitions.
6 Combine	[1,2,5,7,8]	Concatenate sorted left, pivot, right.

EXPLAIN THE METHOD

The pivot must finish between values that satisfy the partition predicate.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define pivot and partition, and state quick sort average-case time complexity.

QUESTION 2 / 5 MARKS

Using first item as pivot, show the conceptual partitions of [6,9,3,8,2] and final sorted array.

QUESTION 3 / 6 MARKS

Explain why quick sort can be O(n log n) on balanced partitions but O(n^2) when partitions are consistently highly unbalanced.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Pivot is the chosen comparison value [1].
- Partition groups values by relation to pivot [1].
- Average time O(n log n) [1].

Q2 / 5 MARKS

- Pivot 6 [1].
- Lower partition [3,2] [1].
- Higher partition [9,8] [1].
- Sort left [2,3] and right [8,9] [1].
- Final [2,3,6,8,9] [1].

Q3 / 6 MARKS

- Balanced partitions give about log n levels [1].
- Each level examines approximately n items [1].
- So O(n log n) [1].
- Unbalanced partitions can give n levels [1].
- Work across levels is n+(n-1)+... [1].
- Worst case O(n^2) [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Explicitly identify pivot choice and partitioning convention.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Dijkstra: shortest known distances

Dijkstra finds shortest paths from one start node when edge weights are non-negative. Choose the unvisited node with the smallest tentative distance and relax its outgoing edges.

DISTANCE

Best known cost so far.

RELAX

Test a route via current node.

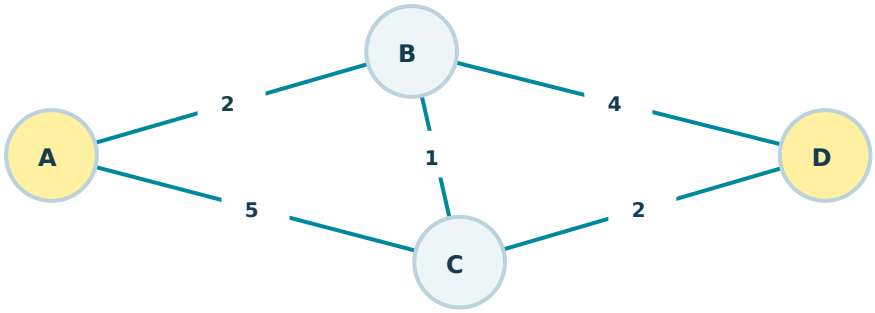
SETTLE

Finalise minimum tentative node.

RULE

No negative edge weights.

HOW THE PARTS FIT TOGETHER



Undirected, non-negative edge costs • diagram is illustrative

VISUAL MEMORY BRIDGE

Imagine spreading a ripple through a road network: the next confirmed location is whichever can be reached at the lowest current cost.

WORDS THAT EARN MARKS

Tentative	Not yet final	**Relaxation**	Improve a bound
Visited	Settled	**Non-negative**	Weight >= 0

EXAM TRAP

Do not mark an unvisited node final just because it was discovered; settle it only when it has the lowest tentative distance.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Find the shortest route from A to D

Undirected positive edges A-B=2, A-C=5, B-C=1, B-D=4, C-D=2.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Initialise	A=0, B=C=D=infinity	Start node has zero cost.
2 Settle A	B=2 via A; C=5 via A	Relax edges leaving A.
3 Settle B	C=min(5,2+1)=3; D=6	B is current cheapest.
4 Settle C	D=min(6,3+2)=5	Improve D via C.
5 Settle D	D=5, predecessor C	Cheapest remaining node is final.
6 Reconstruct	A -> B -> C -> D; cost 5	Follow predecessors backwards.

EXPLAIN THE METHOD

A tentative distance may decrease several times before its vertex is settled.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State the weight restriction and two central actions in Dijkstra's algorithm.

QUESTION 2 / 5 MARKS

In the given graph, show how tentative distance to C and D change after settling B and then C.

QUESTION 3 / 6 MARKS

Trace shortest distance and predecessor chain from A to D; explain why a direct A-C route is not optimal.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- All edge weights are non-negative [1].
- Select lowest tentative unvisited vertex [1].
- Relax outgoing edges / settle selected vertex [1].

Q2 / 5 MARKS

- After A, C=5 [1].
- After B, C becomes 3 via B [1].
- After B, D becomes 6 via B [1].
- After C, D improves to 5 [1].
- Predecessor for D becomes C [1].

Q3 / 6 MARKS

- A=0 [1], B=2 [1], C=3 via B [1], D=5 via C [1].
- Path A-B-C-D [1].
- Direct A-C costs 5 versus A-B-C costs 3, so it is not cheapest for reaching C [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep tentative distances and finalised vertices in separate columns.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

A*: use a lower-bound heuristic

A* combines known cost g(n) with an estimated remaining cost h(n): f(n)=g(n)+h(n). With an admissible heuristic and appropriate search implementation it can find an optimal route.

g(n)

Cost from start to current node.

h(n)

Estimate of cost to goal.

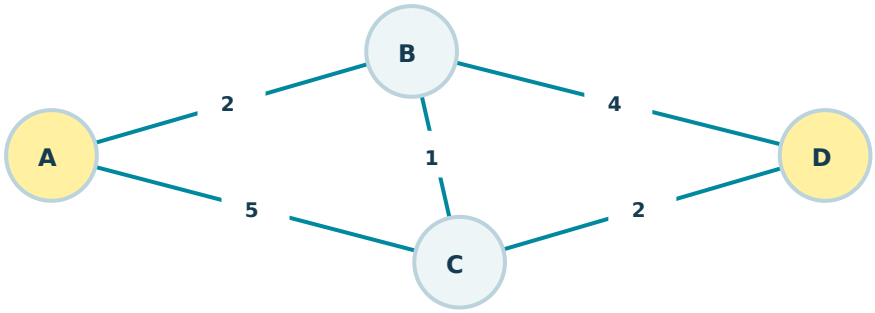
f(n)

Priority g+h.

ADMISSIBLE

Heuristic never overestimates remaining cost.

HOW THE PARTS FIT TOGETHER



Undirected, non-negative edge costs • diagram is illustrative

VISUAL MEMORY BRIDGE

Driving with a map: actual distance driven is g; a safe lower-bound estimate to the destination is h; f helps decide which route to examine next.

WORDS THAT EARN MARKS

Heuristic	Estimate	**Admissible**	No overestimate
g-score	Known cost	**f-score**	g + h

EXAM TRAP

A* chooses the smallest f, not necessarily the smallest g; careless overestimates can lose the optimal-path guarantee.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Compare A* candidates

At a search step, B has g=2,h=2; C has g=3,h=1; D has g=6,h=0. Lower f is preferred.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 B	f=2+2=4	B has optimistic total cost 4.
2 C	f=3+1=4	C has equal priority.
3 D	f=6+0=6	Goal is discovered but not yet best.
4 Select	Choose B or C (tie rule declared)	Priority 4 is lowest.
5 Improve	Via C, D may drop to g=5	Update goal candidate.
6 Explain	f estimates full route cost	g measured, h estimated.

EXPLAIN THE METHOD

Record all three values for each candidate; a discovered goal may not yet be the best solution.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define g, h and f in A* and give the formula relating them.

QUESTION 2 / 5 MARKS

Calculate f for B (g=2,h=2), C (g=3,h=1) and D (g=6,h=0). Which candidate(s) have first priority?

QUESTION 3 / 6 MARKS

Explain admissibility and compare A* with Dijkstra in terms of use of an estimate, ordering and optimality assumptions.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- g is accumulated known path cost [1].
- h is estimated remaining cost [1].
- f = g+h [1].

Q2 / 5 MARKS

- B: f=4 [1].
- C: f=4 [1].
- D: f=6 [1].
- B and C tie for smallest f [1].
- Tie-break should be declared / either candidate may be selected [1].

Q3 / 6 MARKS

- Admissible h never overestimates true remaining cost [1].
- A* orders by g+h [1].
- Dijkstra effectively uses g alone (h=0) [1].
- Dijkstra requires non-negative edges [1].
- With appropriate admissible/consistent heuristic handling A* can be optimal [1].
- A* can explore fewer nodes when heuristic gives useful guidance [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep g and h distinct, and never label a heuristic as guaranteed accurate.

2.3.1(A-F) / SEE IT / CONCEPT AND VISUAL

Synoptic algorithm decisions

A-level solutions require more than naming an algorithm: state its input requirements, trace its behaviour, justify time/space, and check boundary cases.

SEARCH

Sorted? How many lookups?

SORT

Data size, order, memory?

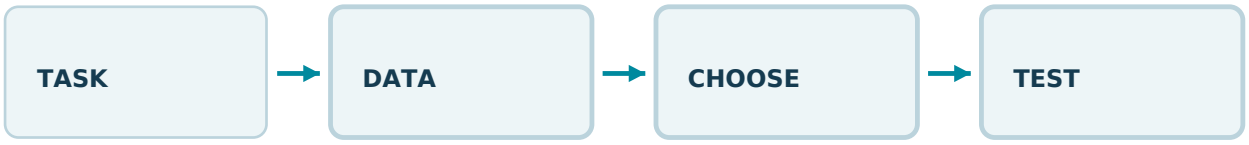
TRAVERSE

Stack/post-order or queue/BFS?

ROUTE

Non-negative weights; heuristic?

HOW THE PARTS FIT TOGETHER



TASK -> DATA PROPERTY -> ALGORITHM -> TRACE -> COMPLEXITY

VISUAL MEMORY BRIDGE

Like choosing tools for a repair: identify the job, inspect the materials, choose a suitable tool, then test whether it actually works.

WORDS THAT EARN MARKS

Justify	Reason with evidence	**Trace**	Track state
Big O	Scalability	**Invariant**	Stays true

EXAM TRAP

Naming a fast algorithm without its prerequisite is an incomplete justification.

2.3.1(A-F) / WORKED EXAMPLE / STEP BY STEP

Choose algorithms for three scenarios

Each scenario changes the data property, required output or resources.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Lookup	1 search in 18 unsorted IDs	Linear search; no setup cost.
2 Lookup	Repeated searches in sorted indexed IDs	Binary search; O(log n).
3 Sorting	Many elements, predictable time	Merge sort O(n log n), extra array.
4 Tree order	Children before their parent	Post-order DFS.
5 Tree order	Visit nearest levels first	BFS with a queue.
6 Routing	Non-negative weights, useful safe h	A* or Dijkstra with assumptions stated.

EXPLAIN THE METHOD

A strong justification links the algorithm to the particular input and identifies at least one limitation.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(A-F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Select a suitable traversal for (i) processing child nodes before their parent, and (ii) exploring a tree level by level, naming the data structure for (ii).

QUESTION 2 / 5 MARKS

A small unsorted list is searched once; a much larger sorted list is searched repeatedly. Recommend and justify a search for each.

QUESTION 3 / 6 MARKS

A navigation system has non-negative road costs and an admissible distance heuristic. Compare Dijkstra with A* and set out two validation cases before deployment.

2.3.1(A-F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Post-order DFS for child-before-parent [1].
- BFS for level-order [1].
- BFS uses a FIFO queue [1].

Q2 / 5 MARKS

- Small unsorted list: linear search [1].
- No sorting prerequisite or cost [1].
- Large sorted list: binary search [1].
- Repeatedly halves candidates / O(log n) [1].
- Sorted prerequisite must be maintained [1].

Q3 / 6 MARKS

- Dijkstra uses lowest tentative g [1].
- A* orders by f=g+h [1].
- A useful admissible heuristic can reduce exploration [1].
- Non-negative edges required for standard Dijkstra [1].
- Test same start and goal / zero-length route [1].
- Test unreachable node or alternative equal-cost paths [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Use the scenario to drive the justification, rather than a memorised description.

OCR APPENDIX 5d / SOURCE-ALIGNED CHECKLIST

What students actually need to do

This supplement makes the official reading conventions and OCR algorithm expectations explicit. It extends the 20 main learning cycles without replacing them.

87	Appendix checklist and exam conventions
88-91	Write searches / OCR zero-based arrays
92-95	Stack, queue, linked list / pointer code
96-99	Tree post-order DFS and BFS / write code
100-103	Bubble and insertion / work and code
104-107	Merge and quick sort / recursion
108-111	Dijkstra and A* / precise traces

SPECIFICATION VS GUIDANCE

Appendix 5d supplies notation for reading code, not a list of compulsory memorised code patterns. OCR subject clarification expands the practical skills: read, trace and write searches, sorts and data structure operations; calculate Dijkstra/A* routes and trace their code.

UNDERSTAND / 1 OF 4

Write searches in OCR-style pseudocode

A Level H446 uses readable pseudocode. Arrays in the OCR examples are zero-indexed; assignment uses =, equality uses == and integer division uses DIV. A clear correct algorithm matters more than memorising exact punctuation.

CORE EXAM EXPECTATIONS

- 0-based array: indexes 0 to n-1
- Use while ... endwhile and if ... then ... endif
- Binary search: best O(1), worst O(log n), sorted data
- When no match: return -1 only after low > high

EXAM CODE / KEY REFERENCE

```
array scores[4]
scores[0] = 5
scores[1] = 9
scores[2] = 12
scores[3] = 20
if scores[2] == 12 then
    print("found")
endif
```

Do not import J277 ERL rules into H446. Correct understandable A Level pseudocode is accepted; OCR’s appendix is the familiar reading style.

MODEL / 2 OF 4

Binary search: fully written example

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
function findSorted(items, n, target)
    low = 0
    high = n - 1
    while low <= high
        mid = (low + high) DIV 2
        if items[mid] == target then
            return mid
        elseif items[mid] < target then
            low = mid + 1
        else
            high = mid - 1
        endif
    endwhile
    return -1
endfunction
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. Given [2,5,8,11,14,17,20], target 5

2. low=0, high=6: mid=3, value=11 -> high=2

3. low=0, high=2: mid=1, value=5 -> return 1

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

State how OCR-style pseudocode assigns a value, compares equality and indexes an array.

QUESTION 2 / 5 MARKS

Trace the mid values and updated bounds when searching for 17 in [2,5,8,11,14,17,20].

QUESTION 3 / 6 MARKS

Write a linear search function for an array items of length n. Return the first matching index, or -1.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: assignment = [1]; comparison == [1]; indices start at 0 [1].

Q2 / 5 MARKS

5 marks: low=0/high=6 [1]; mid=3 [1], value=11 [1]; low=4 [1]; mid=5 and value=17, return 5 [1].

Q3 / 6 MARKS

6 marks: loop from index 0 [1] through n-1 [1]; compare items[i] to target [1]; return matching index immediately [1]; return -1 after loop [1]; correct function/termination structure [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
function findLinear(items, n, target)
    for i = 0 to n - 1
        if items[i] == target then
            return i
        endif
    next i
    return -1
endfunction
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Implement stack, queue and linked-list operations

OCR expects learners to read, trace and write code that adds and removes values. Show pointer changes and error cases, not just the LIFO/FIFO definitions.

CORE EXAM EXPECTATIONS

- For a stack, top points to the next free array slot
- Queue front = next to remove; rear = next free slot
- Check full/empty before changing pointers
- Linked-list insertion: new.next first, then previous.next

EXAM CODE / KEY REFERENCE

```
// Stack of capacity 4, initially top = 0
array stack[4]
// Push 9: stack[top] = 9; top = top + 1
// Pop: top = top - 1; return stack[top]
// Empty when top == 0; full when top == 4
```

A pointer update is not a value deletion: explain which node becomes reachable and which reference changes. State queue pointer conventions.

MODEL / 2 OF 4

Safe operations with explicit pointers

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
function push(value)
    if top == capacity then return false endif
    stack[top] = value
    top = top + 1
    return true
endfunction
function pop()
    if top == 0 then return "EMPTY" endif
    top = top - 1
    return stack[top]
endfunction
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. top=0 -> push(4): stack[0]=4, top=1
2. push(7): stack[1]=7, top=2
3. pop(): top=1, returns 7; 4 remains
4. List insertion after 4: new.next=oldNext; node4.next=new

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

Explain what top represents and identify one stack overflow and underflow condition.

QUESTION 2 / 5 MARKS

Trace push(4), push(7), pop(), push(2) with top initially 0. State the return and final logical stack.

QUESTION 3 / 6 MARKS

Describe an array circular queue with front, rear and count; give safe enqueue/dequeue pointer updates and error checks.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: next free slot [1]; overflow top==capacity [1]; underflow top==0 [1].

Q2 / 5 MARKS

5 marks: push4 gives top1 [1], push7 gives top2 [1], pop returns7 [1], top falls to1 [1], push2 leaves [4,2] [1].

Q3 / 6 MARKS

6 marks: reject enqueue when count==capacity [1]; write q[rear] [1]; rear=(rear+1) MOD capacity [1]; reject dequeue when count==0 [1]; read q[front], then front=(front+1) MOD capacity [1]; increment/decrement count appropriately [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
// Singly linked insertion after current
newNode.next = current.next
current.next = newNode
// Deleting after current
if current.next != NULL then
    current.next = current.next.next
endif
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Write post-order DFS and queue-based BFS

The required OCR tree traversal is specifically depth-first POST-ORDER, not generic preorder. BFS uses a queue to visit each level before its children.

CORE EXAM EXPECTATIONS

- Post-order: left, right, root
- BFS: use FIFO queue
- Each node in a tree is processed once: O(n) time
- For recursive DFS, recursion depth depends on tree height

EXAM CODE / KEY REFERENCE

```
// Tree: root 8; children 3 and 12
// 3 has children 1 and 6
// 12 has children 10 and 14
// Post-order: 1,6,3,10,14,12,8
// Breadth-first: 8,3,12,1,6,10,14
```

Do not say “DFS” alone when the question demands post-order; explicit left/right/root order earns the explanation.

MODEL / 2 OF 4

Two complete traversal outlines

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
procedure post(node)
    if node != NULL then
        post(node.left)
        post(node.right)
        print(node.value)
    endif
endprocedure

procedure breadth(root)
    q = new Queue()
    if root != NULL then q.enqueue(root) endif
    while NOT q.isEmpty()
        node = q.dequeue()
        print(node.value)
        for child in node.children
            q.enqueue(child)
        next child
    endwhile
endprocedure
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. BFS queue after visiting 8: [3,12]
2. After visiting 3: [12,1,6]
3. After visiting 12: [1,6,10,14]
4. Post-order does not print a parent until both children finish.

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

Explain why a FIFO queue produces breadth-first rather than depth-first traversal.

QUESTION 2 / 5 MARKS

Produce both orders for tree A with children B,C; B has children D,E, and C has child F.

QUESTION 3 / 6 MARKS

Write recursive post-order pseudocode and explain its termination and time complexity.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: enqueue discovered children [1]; remove oldest pending node [1]; therefore visit current level before newly enqueued descendants [1].

Q2 / 5 MARKS

5 marks: post-order D,E,B,F,C,A [3]; BFS A,B,C,D,E,F [2].

Q3 / 6 MARKS

6 marks: null/base guard [1]; left recursive call [1]; right recursive call [1]; root output after children [1]; terminates at null leaves [1]; O(n) because each node processed once [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
procedure postOrder(node)
    if node != NULL then
        postOrder(node.left)
        postOrder(node.right)
        print(node.value)
    endif
endprocedure
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Read and write bubble and insertion sort

Exam tasks may show incomplete code or a list after each pass. State whether the bubble implementation has an early-exit check before quoting its best case.

CORE EXAM EXPECTATIONS

- Bubble: compare adjacent values and swap if left > right
- One completed bubble pass fixes the largest remaining value
- Insertion: save key, shift larger items right
- Both worst-case $O(n^2)$; best $O(n)$ for insertion and optimised bubble

EXAM CODE / KEY REFERENCE

```
// One bubble pass over [5,1,4,2]
[5,1,4,2] -> [1,5,4,2]
           -> [1,4,5,2]
           -> [1,4,2,5]
// Largest item 5 now fixed at the end
```

A missing temporary variable can overwrite a value. Distinguish passes, comparisons and swaps.

MODEL / 2 OF 4

Insertion sort with 0-based array and key

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
procedure insertionSort(a, n)
    for i = 1 to n - 1
        key = a[i]
        j = i - 1
        while j >= 0 AND a[j] > key
            a[j+1] = a[j]
            j = j - 1
        endwhile
        a[j+1] = key
    next i
endprocedure
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. [5,2,4,1]: insert 2 -> [2,5,4,1]
2. Insert 4 -> [2,4,5,1]
3. Insert 1 -> [1,2,4,5]
4. Only items greater than the key shift.

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

State the adjacent swap rule in bubble sort and how an early-exit version can terminate.

QUESTION 2 / 5 MARKS

Show array states after each insertion of [4,2,3,1] in insertion sort.

QUESTION 3 / 6 MARKS

Write the critical inner bubble-sort loop with a swap and explain its worst-case time complexity.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: compare adjacent items [1]; swap if left>right [1]; end when a complete pass makes no swaps [1].

Q2 / 5 MARKS

5 marks: after key2 [2,4,3,1] [1]; key3 [2,3,4,1] [2]; key1 [1,2,3,4] [2].

Q3 / 6 MARKS

6 marks: loop across adjacent pairs [1]; compare > [1]; swap correctly with temp [2]; repeat for passes [1]; $O(n^2)$ because passes contain a linear number of comparisons [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
// A bubble pass on indices 0 .. end-1
for j = 0 to end - 1
    if a[j] > a[j+1] then
        temp = a[j]
        a[j] = a[j+1]
        a[j+1] = temp
    endif
next j
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Merge sort and quick sort

Show what each recursive step does, including the stopping condition. Helpers below have precise contracts; they are not built-in OCR commands.

WHAT THE ALGORITHMS DO

MERGE SORT

Split into halves, recursively sort each half, then merge the two already-sorted halves. Standard array version: O(n log n) time; O(n) auxiliary buffer.

QUICK SORT

Choose a pivot, put lower/equal/higher values into three partitions and recurse on lower and higher only. Average O(n log n); worst O(n^2) if pivots remain highly unbalanced.

HELPER CONTRACTS / ALL INDEXES ZERO-BASED

copyRange(a,start,stop)

Returns a new array from indexes start to stop-1; stop is EXCLUSIVE. Example: copyRange([7,2,6,1],0,2) -> [7,2].

mergeSorted(left,right)

Takes two sorted inputs and repeatedly takes the lower front element; copies the remaining tail when one input is exhausted.

partition3(a,pivot)

Returns (less,equal,more). Every input item appears exactly once; ALL values equal to pivot go to equal, avoiding recursive loops on duplicate pivots.

QUICK VISUAL CHECK

Merge [2,7] + [1,6] -> [1,2,6,7]. Pivot 6: [6,9,3,8,2,6] -> [3,2] | [6,6] | [9,8].

MODEL / 2 OF 4

Recursive structure (helpers defined)

Read each helper contract on the previous page. The pseudocode below is an algorithm outline using those named operations, not a verbatim OCR Appendix 5d language requirement.

SORTING ROUTINES / END-EXCLUSIVE SLICES

```
function mergeSort(a)
    if a.length <= 1 then return a endif
    mid = a.length DIV 2
    left = mergeSort(copyRange(a,0,mid))
    right = mergeSort(copyRange(a,mid,a.length))
    return mergeSorted(left,right)
endfunction

function quickSort(a)
    if a.length <= 1 then return a endif
    pivot = a[0]
    less,equal,more = partition3(a,pivot)
    return quickSort(less) + equal + quickSort(more)
endfunction
```

WORKED TRACE / INCLUDE EACH INTERMEDIATE STATE

MERGE [8,3,5,2]

[8,3] + [5,2] -> [8],[3],[5],[2] -> [3,8] + [2,5] -> [2,3,5,8].

QUICK [6,9,3,8,2]

Pivot 6 -> less [3,2], equal [6], more [9,8] -> [2,3] + [6] + [8,9].

QUICK WITH DUPLICATES

[6,9,3,8,2,6] puts both 6s into equal, so recursive sublists shrink; result [2,3,6,6,8,9].

TRY / 3 OF 4

Trace, write, explain

Attempt all three original exam-style questions in black pen. The marks are indicative; do not turn over until you have completed your answer.

BLACK PEN / ANSWER BEFORE CHECKING

QUESTION 1 / 3 MARKS

State the base case, what must be true of the inputs to a merge step, and the extra array-space cost of standard merge sort.

QUESTION 2 / 5 MARKS

Trace merge sort on [4,1,3,2], showing every split and both intermediate merges before the final result.

QUESTION 3 / 6 MARKS

For first-item-pivot quick sort, describe the partitions of [4,7,2,6,1] and explain how unbalanced pivots affect time complexity.

CHECK / 4 OF 4

Self-mark, correct, explain

Award each point only when it is supported by the student response. These are teacher-written indicative points, not an official OCR mark scheme.

GREEN PEN / INDICATIVE MARKING

Q1 / 3 MARKS

- Base case: zero/one item needs no further sorting [1].
- Merge consumes inputs that are each already in sorted order [1].
- Standard array merge sort uses O(n) auxiliary storage [1].

Q2 / 5 MARKS

- Split [4,1] and [3,2] [1]; then [4],[1],[3],[2] [1].
- Merge left [1,4] [1]; merge right [2,3] [1].
- Final merge produces [1,2,3,4] [1].

Q3 / 6 MARKS

- Choose pivot 4 [1]; lower partition [2,1] [1].
- Higher partition [7,6] [1]; recursively sort both [1].
- Persistent unbalanced pivots give O(n) levels [1].
- Summed comparisons yield O(n^2) worst case [1].

COMPLETE MERGE LOOP / HOW TO HANDLE LEFTOVERS

```
i = 0; j = 0; result = []
while i < left.length AND j < right.length
    if left[i] <= right[j] then
        result.append(left[i]); i = i + 1
    else
        result.append(right[j]); j = j + 1
    endif
endwhile
while i < left.length
    result.append(left[i]); i = i + 1
endwhile
while j < right.length
    result.append(right[j]); j = j + 1
endwhile
// result contains the full sorted merge
```

NOTE: result is a resizable list; [] and append are illustrative pseudocode.

UNDERSTAND / 1 OF 4

Work a Dijkstra and A* route trace

OCR clarification emphasises calculating routes and tracing code. Dijkstra settles lowest tentative distance; A* prioritises f=g+h. Use valid non-negative edge costs and a safe heuristic.

CORE EXAM EXPECTATIONS

- Graph: A-B=2, A-C=5, B-C=1, B-D=4, C-D=2
- Correct shortest route A-B-C-D, total cost 5
- Dijkstra: settle smallest tentative g
- A*: f=g+h; admissible h does not overestimate

EXAM CODE / KEY REFERENCE

```
h(A)=5, h(B)=3, h(C)=2, h(D)=0
// All these h values are true lower bounds
// At A: B g=2 f=5; C g=5 f=7
// After B: C g=3 f=5; D g=6 f=6
// After C: D g=5 f=5
```

The model uses an admissible and consistent heuristic, with goal termination on removal from the open set. “First found goal” is not enough.

MODEL / 2 OF 4

Dijkstra settled-set and A* priority trace

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
// Dijkstra: distances from A
initial: A=0; B=C=D=infinity
settle A: B=2 via A; C=5 via A
settle B: C=3 via B; D=6 via B
settle C: D=5 via C
settle D: route A-B-C-D, cost=5

// A* candidate priority f = g + h
after A: B=2+3=5; C=5+2=7
after B: C=3+2=5; D=6+0=6
after C: D=5+0=5; pop goal D
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. Key update: compare old C=5 against route A-B-C=2+1=3.
2. D initially 6 via B, improves to 5 via C.
3. With h as above, A* chooses B, then C, then D.
4. A* must not stop merely when a costly goal is first discovered.

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

Define g, h and f in A*, giving the formula for f.

QUESTION 2 / 5 MARKS

Trace Dijkstra from A to D: record the order settled and each improvement of C and D.

QUESTION 3 / 6 MARKS

Calculate A* f priorities after A, B and C with the supplied h; explain why the final result is optimal under these assumptions.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: g=known start-to-node cost [1], h=estimated remaining cost [1], f=g+h [1].

Q2 / 5 MARKS

5 marks: settle A,B,C,D [1]; B=2 [1]; C 5->3 [1]; D 6->5 [1]; path A-B-C-D cost5 [1].

Q3 / 6 MARKS

6 marks: B f=5 and C f=7 after A [2]; after B C f=5 and D f=6 [2]; after C D f=5 [1]; admissible (indeed consistent) h, non-negative edges and correct open-list handling preserve optimality [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
// A* open-list comparison (outline)
while openSet is not empty
    current = itemWithSmallestF(openSet)
    if current == goal then return path endif
    for each neighbour of current
        candidateG = g[current] + edgeCost
        if candidateG < g[neighbour] then
            predecessor[neighbour] = current
            g[neighbour] = candidateG
            f[neighbour] = candidateG + h[neighbour]
            addOrUpdate(openSet, neighbour)
        endif
    next neighbour
endwhile
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.